

PROJECT CLASSIFICATION: PORTFOLIO — FLAGSHIP

Industrial Motor Control and Protection System

Engineering Execution Plan

STM32G474 · FreeRTOS · CAN · PID · Fault Management · PCB Design

Document Type	Engineering Execution Plan — Full Scope
Revision	1.0 — Initial Release
Issued By	Project Engineering Office
Target Role	Embedded Firmware Engineer — Entry Level
MCU Platform	STM32G474RET6
RTOS	FreeRTOS v10.x
Target Markets	Industrial, Automotive (EV), Medical, Robotics
Issue Date	June 08, 2026

This document is the authoritative engineering execution plan for the Industrial Motor Control and Protection System portfolio project. It defines project scope, architecture, phased deliverables, component specifications, firmware design, validation requirements, and documentation standards. All engineering decisions shall be traceable to requirements defined herein.

TABLE OF CONTENTS

1.	Project Overview and Objectives	3
2.	Success Criteria and Exit Conditions	4
3.	System Architecture	5
4.	Component Specification and Justification	6
5.	Firmware Architecture	8
6.	Phased Execution Plan	10
7.	RTOS Design and Task Specification	15
8.	Driver Development Specification	17
9.	PID Control System Design	19
10.	CAN Protocol Specification	20
11.	Fault Management System	21
12.	Validation and Test Plan	22
13.	Debug Evidence Requirements	23
14.	Risk Register	24
15.	Final Deliverables Checklist	25

1. PROJECT OVERVIEW AND OBJECTIVES

PROJECT MISSION

This project exists to produce a single, flagship portfolio artifact that demonstrates the full breadth of embedded firmware engineering competency at a level consistent with a junior-to-mid-level industry role. The system must be buildable, testable, debuggable, and documentable in a manner indistinguishable from a professional commercial development effort. Cutting corners on any of these dimensions defeats the purpose of the project.

1.1 Project Description

The Industrial Motor Control and Protection System is a closed-loop DC motor controller built around the STM32G474 microcontroller running FreeRTOS. The system accepts speed commands via a local rotary encoder interface or remotely via CAN bus, executes a PID control algorithm to regulate motor RPM, monitors current and voltage in real time, detects and logs fault conditions to SPI flash memory, and presents system status on a local OLED display.

The project deliberately mirrors the architecture and engineering discipline expected in industrial, automotive, and medical embedded product development: layered firmware, defined interfaces between software modules, formal fault handling with safe-state behavior, hardware-in-the-loop validation, and complete engineering documentation.

1.2 Engineering Objectives

ID	Category	Objective Description
OBJ-01	Control	Demonstrate closed-loop PID speed regulation of a DC motor across load conditions.
OBJ-02	Sensing	Acquire and display current, voltage, temperature, and RPM with calibrated accuracy.
OBJ-03	RTOS	Implement a multi-task FreeRTOS architecture with defined priorities, periods, and inter-task communication.
OBJ-04	Networking	Implement CAN communication using the MCP2515 controller; accept and respond to motor command frames.
OBJ-05	Fault Mgmt	Detect, classify, and log fault conditions; execute safe-state behavior on fault activation.
OBJ-06	Storage	Log timestamped fault and event records to W25Q128 SPI flash with structured formatting.
OBJ-07	UI	Provide a functional local OLED menu system navigated by rotary encoder.
OBJ-08	Debug	Collect oscilloscope, logic analyzer, and JTAG debug evidence during validation.
OBJ-09	PCB	Produce a professional two-layer PCB schematic and layout in KiCad.
OBJ-10	Docs	Deliver a complete documentation package equivalent to a professional product release.

1.3 Target Market Alignment

The technical content of this project is intentionally designed to produce maximum portfolio value across multiple embedded hiring markets. The table below maps project elements to their market relevance.

Target Market	Relevant Project Elements	Alignment
Industrial Automation	CAN networking, fault management, event logging, RTOS multi-tasking, motor control	Very High
Robotics	Closed-loop PID, encoder feedback, real-time task scheduling, sensor fusion	Very High
Automotive (EV/ADAS)	CAN protocol, STM32 platform, structured fault handling, safe-state design	High
Medical Devices	Fault detection, safe shutdown, event audit logging, deterministic RTOS design	High
Consumer Electronics	RTOS, driver development, SPI/I2C peripherals, embedded UI	Medium

2. SUCCESS CRITERIA AND EXIT CONDITIONS

The following criteria define the minimum acceptable state for project completion. The project shall not be considered complete for portfolio presentation until all REQUIRED criteria are met. RECOMMENDED criteria significantly strengthen the portfolio presentation and should be pursued.

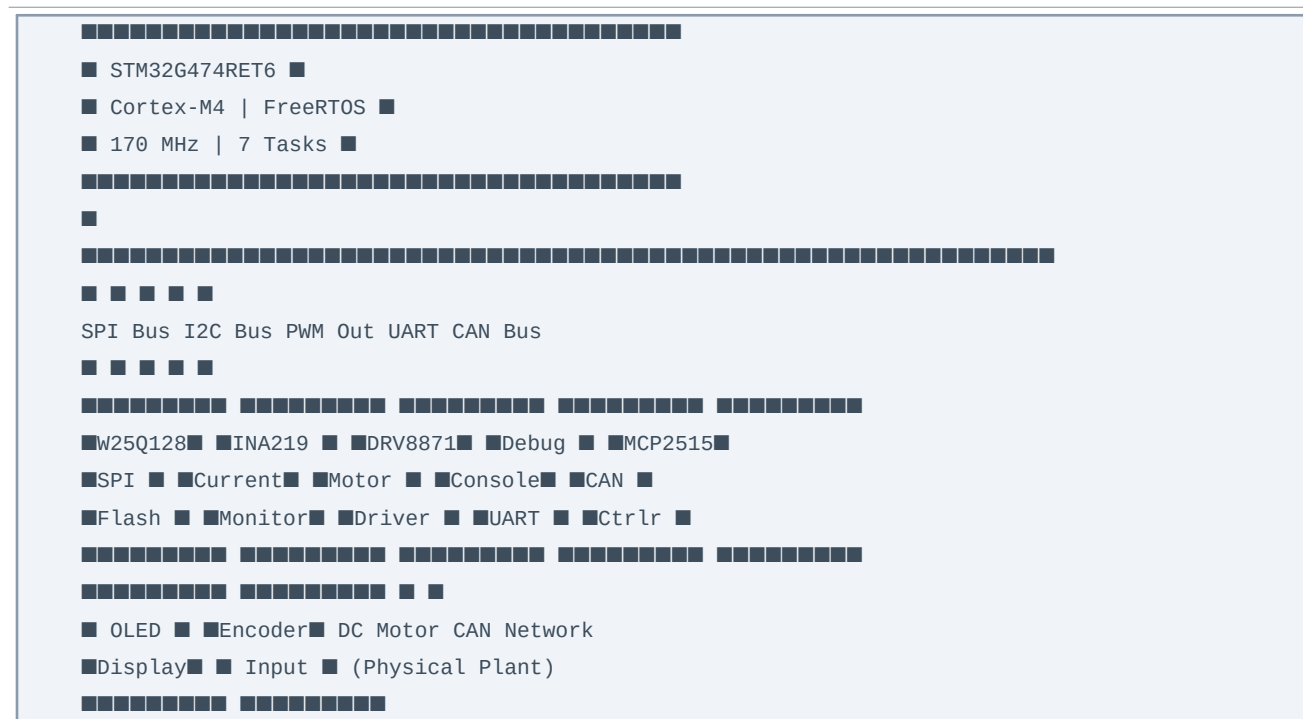
ID	Criterion	Status
SC-01	Motor starts, runs, and stops on command via encoder UI	REQUIRED
SC-02	Motor reaches commanded RPM within $\pm 5\%$ under no-load conditions	REQUIRED
SC-03	PID controller maintains RPM under applied load without steady-state error	REQUIRED
SC-04	Current, voltage, and RPM displayed live on OLED with <200ms update latency	REQUIRED
SC-05	Overcurrent fault detected, logged to flash, and motor stopped within 100ms	REQUIRED
SC-06	Sensor timeout fault detected and safe shutdown executed	REQUIRED
SC-07	CAN command received and motor responds correctly	REQUIRED
SC-08	Fault log readable and timestamped entries retrievable via UART console	REQUIRED
SC-09	Oscilloscope captures of PWM, startup current, and current limiting event collected	REQUIRED
SC-10	Logic analyzer captures of SPI, I2C, and CAN transactions collected	REQUIRED
SC-11	JTAG session demonstrating task inspection and live variable watch collected	REQUIRED
SC-12	Full demonstration sequence completed and recorded as video	REQUIRED
SC-13	All documentation deliverables completed	REQUIRED
SC-14	PCB schematic completed and reviewed in KiCad	REQUIRED
SC-15	PCB layout completed with design rule check passing	RECOMMENDED
SC-16	PCB manufactured and used as hardware platform for final demo	RECOMMENDED
SC-17	Second CAN node (supervisory controller) implemented and demonstrated	RECOMMENDED
SC-18	MISRA-C coding guideline violations documented and justified	RECOMMENDED

3. SYSTEM ARCHITECTURE

3.1 System Overview

The system is organized as a hub-and-spoke architecture centered on the STM32G474 MCU. All peripherals connect via standard serial buses (SPI, I2C) or direct MCU signals (PWM, GPIO). The firmware stack is organized into four distinct horizontal layers with strict upward-only dependency rules: no lower layer may reference a higher layer.

3.2 System Block Diagram



3.3 Communication Bus Summary

Bus / Interface	Speed	Connected Devices / Purpose	MCU Peripheral
SPI (3-wire + CS)	18 MHz	W25Q128 Flash, MCP2515 CAN Controller, OLED Display	SPI1, SPI2
I2C	400 kHz	INA219 Current/Voltage Monitor	I2C1
PWM	20 kHz	DRV8871 Motor Driver — duty cycle = speed command	TIM1 CH1
UART	115200	Debug console, CLI, fault log retrieval	USART2
CAN	500 kbps	MCP2515 external CAN controller (SPI bridge)	SPI2 + INT GPIO
GPIO	N/A	Rotary encoder (A/B/SW), fault LED, motor EN, INT lines	Various

4. COMPONENT SPECIFICATION AND JUSTIFICATION

All components were selected to reflect real industrial design decisions rather than hobbyist convenience. Each selection is justified below with the engineering rationale a technical interviewer would expect you to be able to articulate.

4.1 STM32G474RET6 — Primary Microcontroller

Package / Core	LQFP-64, Cortex-M4F, 170 MHz, FPU, DSP instructions
Flash / RAM	512 KB Flash, 128 KB SRAM
Key Peripherals	3× SPI, 2× I2C, 3× UART, Advanced Timer (TIM1) for center-aligned PWM, FDCAN, 12-bit ADC
Justification	The G474 is targeted at motor control and power conversion applications. Center-aligned PWM on TIM1 eliminates shoot-through risk. FDCAN provides native CAN support (supplemented by MCP2515 for hardware simplicity). STM32 is ubiquitous in industrial embedded roles — HAL familiarity transfers directly.
Datasheet	DS12288 Rev 4 — STM32G474xB/xC/xE

4.2 INA219 — Current and Voltage Monitor

Interface	I2C, configurable address (up to 4 devices per bus)
Range	0–3.2 A current, 0–32 V bus voltage, 12-bit resolution
Justification	Industry-standard current sense IC used in power management across automotive and industrial applications. High-side current sensing avoids ground disruption. I2C interface keeps SPI bus bandwidth free. Demonstrates understanding of shunt-based current measurement principles.
Note	Procure x2 units. Configure for 10-mΩ shunt resistor. Calibrate per datasheet §8.5.

4.3 MCP2515 — CAN Controller

Interface	SPI up to 10 MHz, MCP2515 drives TJA1050 CAN transceiver
CAN Standard	CAN 2.0B, supports standard (11-bit) and extended (29-bit) identifiers
Justification	Using an external SPI-bridged CAN controller rather than direct FDCAN peripheral demonstrates understanding of real-world board design constraints: the MCP2515 is common on designs where CAN pin placement drives routing decisions. It also requires explicit SPI driver implementation, which is a stronger portfolio signal than a HAL abstraction.
Note	Pair with TJA1050 or SN65HVD230 transceiver. Requires 120Ω termination resistor at each bus end.

4.4 W25Q128 — SPI NOR Flash

Capacity	128 Mbit (16 MB), 256-byte page program, 4 KB sector erase
----------	--

Interface	SPI / QSPI up to 80 MHz
Justification	Persistent fault and event logging requires non-volatile storage that survives power cycles. EEPROM is too small; SD cards add mechanical complexity. NOR flash is the standard choice for embedded event logs. Implementing a lightweight wear-leveled log structure on raw flash demonstrates low-level storage engineering skills.
Note	Design log as circular buffer with sector-erase boundary awareness. Minimum 100,000 erase cycles per sector.

4.5 DRV8871 — Motor Driver

Rating	3.6 A peak, 2.5 A RMS, 6.5–45 V operating voltage
Interface	Two PWM inputs (IN1/IN2), built-in current limiting, nFAULT output
Justification	The DRV8871 integrates a full H-bridge with current limiting, eliminating discrete MOSFET design complexity while still requiring correct dead-time and thermal management analysis. The nFAULT pin provides a hardware signal that can interrupt the MCU independently of software — a critical safety feature.
Warning	Verify motor stall current does not exceed 3.6 A peak with margin. Add thermal relief copper pours around IC on PCB.

4.6 128×64 OLED Display (SSD1306)

Interface	SPI or I2C (use SPI for bandwidth)
Justification	Local human interface without dependency on a host computer. Forces implementation of a real embedded menu framework — a common interview topic. Demonstrates graphics buffer management and display driver implementation.

4.7 Rotary Encoder with Push Button

Type	Incremental, quadrature output, integrated switch
Interface	GPIO interrupt-driven quadrature decode or TIM encoder mode
Justification	Primary user input device. TIM encoder mode offloads quadrature decoding to hardware and demonstrates peripheral knowledge. Push button provides menu confirm / fault clear.

5. FIRMWARE ARCHITECTURE

5.1 Layered Architecture

The firmware shall be organized into four strictly separated horizontal layers. Dependencies flow upward only — upper layers call lower layers; lower layers never reference upper layers. This separation enables unit testing of individual layers, simplifies debugging, and mirrors professional firmware product structure.

Layer	Responsibilities	Dependency Rule
4 — Application	State machine, menu system, motor control logic, system initialization	References Services layer only. No direct driver calls.
3 — Services	PID controller, fault manager, event logger, telemetry manager	References Driver layer only. Provides clean APIs to Application layer.
2 — Drivers	MCP2515, INA219, OLED, W25Q128 Flash, Encoder	References BSP layer only. Each driver is a standalone module with header interface.
1 — BSP	SPI, I2C, UART, PWM, GPIO, CAN peripheral init and low-level transactions	References STM32 HAL only. No business logic. Thin wrappers around HAL calls.
0 — HAL	STM32 Hardware Abstraction Layer (STM32CubeMX generated)	Generated code. Minimal manual modification. Regeneration must not break upper layers.

5.2 Module Dependency Map

The following defines allowed include relationships between modules:

```
Application Layer:
main.c / system_init.c → state_machine.h, motor_control.h, menu.h
state_machine.c → fault_manager.h, motor_control.h, telemetry.h
motor_control.c → pid.h, telemetry.h

Services Layer:
pid.c → (no external deps – pure math)
fault_manager.c → event_logger.h, ina219.h, encoder.h
event_logger.c → flash.h
telemetry.c → ina219.h, encoder.h, motor_control.h

Driver Layer:
mcp2515.c → bsp_spi.h
ina219.c → bsp_i2c.h
oled.c → bsp_spi.h
flash.c → bsp_spi.h
encoder.c → bsp_gpio.h

BSP Layer:
bsp_spi.c / bsp_i2c.c / bsp_uart.c / bsp_pwm.c / bsp_gpio.c → stm32g4xx_hal.h
```

5.3 Repository Folder Structure

```
industrial-motor-controller/  
  ■■■ Core/  
  ■ ■■■ Inc/ # STM32CubeMX generated headers  
  ■ ■■■ Src/ # STM32CubeMX generated source + main.c  
  ■■■ BSP/  
  ■ ■■■ bsp_spi.c / .h  
  ■ ■■■ bsp_i2c.c / .h  
  ■ ■■■ bsp_uart.c / .h  
  ■ ■■■ bsp_pwm.c / .h  
  ■ ■■■ bsp_gpio.c / .h  
  ■■■ Drivers/  
  ■ ■■■ mcp2515.c / .h  
  ■ ■■■ ina219.c / .h  
  ■ ■■■ oled.c / .h  
  ■ ■■■ flash.c / .h  
  ■ ■■■ encoder.c / .h  
  ■■■ Services/  
  ■ ■■■ pid.c / .h  
  ■ ■■■ fault_manager.c / .h  
  ■ ■■■ event_logger.c / .h  
  ■ ■■■ telemetry.c / .h  
  ■■■ App/  
  ■ ■■■ state_machine.c / .h  
  ■ ■■■ motor_control.c / .h  
  ■ ■■■ menu.c / .h  
  ■■■ RTOS/  
  ■ ■■■ FreeRTOSConfig.h  
  ■■■ Docs/  
  ■ ■■■ architecture/  
  ■ ■■■ validation/  
  ■ ■■■ debug/  
  ■ ■■■ can_protocol/  
  ■■■ Tools/  
  ■■■ can_simulator/ # PC-side Python CAN node
```

6. PHASED EXECUTION PLAN

The project is organized into five sequential phases. Each phase has defined entry conditions, deliverables, and exit criteria. A phase must meet its exit criteria before the next phase begins. This structure ensures a demonstrable, stable baseline exists at all times — if work pauses at any phase boundary, a coherent portfolio artifact still exists.

Important Note on Timeline

The timelines listed per phase are estimates for part-time effort (~2–3 hours/day). Full-time focused effort may compress these significantly.
Do not sacrifice correctness for schedule. A fully working Phase 3 is more valuable than a broken Phase 5.

Phase 1: Foundation and Hardware Bringup

Estimated Duration: 1–2 weeks		Entry Condition: Clean development environment. All ICs and development boards procured and inventoried.
Task	Title	Description
1.1	Environment Setup	Install and configure all required tooling: STM32CubeIDE, STM32CubeMX, KiCad, OpenOCD/J-Link, git. Create the GitHub repository with the folder structure defined in Section 5.3. Initialize with a README describing the project at a professional level.
1.2	STM32CubeMX Configuration	Configure the STM32G474 project: enable SPI1 (flash, OLED), SPI2 (MCP2515), I2C1 (INA219), USART2 (debug), TIM1 (PWM center-aligned, 20 kHz), TIM2 (encoder mode), GPIO for encoder, LEDs, and interrupt lines. Generate HAL code. Verify code compiles and links cleanly.
1.3	UART Debug Console	Implement a minimal UART console in the BSP layer. Verify printf-style output redirected to USART2. Confirm character echo and timestamp output. This becomes the primary debug channel for all subsequent phases.
1.4	FreeRTOS Integration	Add FreeRTOS to the project (via CubeMX or manual integration). Create a single blinky task to verify the scheduler runs. Confirm task stack usage with uxTaskGetStackHighWaterMark. Document stack sizes chosen and rationale.
1.5	Hardware Bench Setup	Wire all ICs on breadboard or protoboard per the schematic. Power the STM32 from ST-Link. Confirm JTAG/SWD connection in STM32CubeIDE debugger. Verify all power rails with multimeter.

Phase Exit Criteria:

- Phase 1 firmware compiles without warnings.
- FreeRTOS scheduler running with at least one task.
- UART console printing timestamp and heartbeat.
- JTAG debug session connects and halts at breakpoints.
- Git repository initialized with clean initial commit.

Phase 2: Peripheral Drivers and Sensor Validation

Estimated Duration: 2–3 weeks		Entry Condition: Phase 1 complete. All hardware wired and powered.
Task	Title	Description
2.1	INA219 Driver	Implement the INA219 driver in the Driver layer (ina219.c / ina219.h). Driver shall expose: INA219_Init(), INA219_ReadCurrent_mA(), INA219_ReadBusVoltage_mV(). Validate readings with a known resistive load and multimeter. Log readings to UART console at 1 Hz. Document calibration register configuration and shunt resistor selection.
2.2	OLED Driver and Display Framework	Implement SSD1306 OLED driver over SPI. Implement a framebuffer approach: all rendering writes to an in-memory 128×64 bit buffer; a single flush function transmits the buffer to the display. Implement primitive functions: draw_pixel, draw_string, draw_line, draw_rect, clear_buffer. Validate by displaying a static screen with project name and version.
2.3	Encoder Driver	Implement encoder driver using STM32 TIM2 encoder mode. Driver shall expose: Encoder_GetPosition(), Encoder_GetDirection(), Encoder_ResetPosition(), Encoder_IsButtonPressed(). Validate position tracking by rotating encoder and observing count in debugger. Debounce button with 20 ms filter.
2.4	Menu System	Implement a hierarchical menu system in the App layer. Menu shall support: top-level status screen, RPM setpoint adjustment, fault log viewing, system info screen. Navigation via encoder rotation (scroll) and button (select/back). Validate full navigation on OLED display.
2.5	Motor Open-Loop Control	Implement DRV8871 PWM control in the BSP/App layers. Generate center-aligned 20 kHz PWM on TIM1 CH1. Implement MotorControl_SetDutyCycle(uint8_t percent). Verify motor spins at commanded duty cycles. Capture PWM waveform on oscilloscope — document in debug log.

Phase Exit Criteria:

- All sensor readings validated against reference measurements.
- OLED displaying live current, voltage, and RPM values.
- Encoder navigation working through all menu screens.
- Motor spinning under open-loop PWM control.
- Oscilloscope capture of PWM waveform saved and committed to Docs/.

Phase 3: Closed-Loop Control

Estimated Duration: 1–2 weeks		Entry Condition: Phase 2 complete. Motor spinning open-loop. RPM measurement functional.
Task	Title	Description
3.1	RPM Measurement	Implement RPM measurement from the motor feedback signal (Hall sensor or optical encoder tachometer, or back-EMF zero-crossing if applicable). Calculate RPM as: $RPM = (pulse_count / pulses_per_rev) \times (60 / measurement_window_seconds)$. Validate by comparing measured RPM against expected at known PWM duty cycles.

Task	Title	Description
3.2	PID Controller Implementation	Implement a discrete-time PID controller in Services/pid.c. The controller shall be fully parameterized: PID_Init(kp, ki, kd, dt, output_min, output_max). Anti-windup shall be implemented via integrator clamping. Derivative filtering via first-order low-pass filter (N=10 typical). See Section 9 for full PID specification.
3.3	PID Tuning	Tune PID gains for stable RPM regulation. Begin with kd=0, ki=0 and increase kp until marginal oscillation (Ziegler-Nichols method), then back off 50% and add integral. Document final gains, step response time, overshoot, and steady-state error in the validation report. Capture RPM step response on oscilloscope via DAC or UART plot.
3.4	Load Disturbance Testing	Apply manual load to motor shaft while PID is active. Verify controller recovers RPM without steady-state error. Verify integrator windup does not cause runaway on large disturbances. Document step load response in validation report.

Phase Exit Criteria:

- Motor maintains commanded RPM within $\pm 5\%$ under no-load.
- PID recovers from manual load disturbance within 2 seconds.
- No integrator windup observed during saturation recovery.
- RPM step response captured and documented.
- PID gains documented with rationale.

Phase 4: Networking, Logging, and Fault Management

Estimated Duration: 2–3 weeks		Entry Condition: Phase 3 complete. Closed-loop control validated.
Task	Title	Description
4.1	MCP2515 CAN Driver	Implement MCP2515 SPI driver. Driver shall expose: MCP2515_Init(), MCP2515_SendFrame(), MCP2515_ReceiveFrame(), MCP2515_SetFilter(). Implement interrupt-driven receive using MCP2515 INT pin → MCU GPIO EXTI. Implement the CAN protocol defined in Section 10. Validate with logic analyzer capture of CAN frames. Validate with second CAN node (PC Python script or second MCU).
4.2	W25Q128 Flash Driver and Event Logger	Implement W25Q128 SPI flash driver: Flash_Init(), Flash_ReadPage(), Flash_WritePage(), Flash_EraseSector(), Flash_ChipErase(). Implement event logger on top of flash driver: circular log buffer, structured 32-byte log records (timestamp, event_type, severity, payload). Implement log retrieval via UART console command. See Section 11 for log record format.
4.3	Fault Manager	Implement fault manager per the specification in Section 11. Fault manager shall monitor: overcurrent (INA219 > threshold), sensor timeout (no I2C response within 500 ms), CAN communication timeout (no frame received within 5 s if CAN mode active), under-voltage (bus voltage < 5.5 V). On any fault: stop motor immediately, set fault LED, log fault event to flash, transition state machine to FAULT state. Require explicit operator reset to clear fault.

Task	Title	Description
4.4	CAN Motor Control Integration	Integrate CAN receive path with motor control. On receipt of MOTOR_CMD frame: validate checksum, extract target RPM, validate range (0–3000 RPM), pass to motor control state machine. Transmit MOTOR_STATUS frame at 100 ms interval regardless of command source.

Phase Exit Criteria:

- CAN command received and motor responds within 200 ms.
- Fault correctly detected, logged, and motor stopped.
- Flash log readable via UART — timestamped entries present.
- Logic analyzer captures of SPI (flash), I2C, and CAN committed to Docs/.
- All fault types tested and documented in validation report.

Phase 5: Validation, Debug Evidence, and Documentation

Estimated Duration: 1–2 weeks		Entry Condition: Phase 4 complete. All features functional.
Task	Title	Description
5.1	Full Demonstration Sequence Dry Run	Execute the complete 15-step demonstration sequence defined in Section 12 three times in sequence without stopping. Identify and resolve any reliability issues. The demo must be repeatable without pre-positioning or manual intervention between steps.
5.2	Debug Evidence Collection	Collect all required debug evidence per Section 13. Each capture shall be saved as an image or logic analyzer export, annotated with labels identifying the signals and events visible, and committed to Docs/debug/.
5.3	PCB Schematic and Layout	Complete the KiCad schematic for the full system. Include: decoupling capacitors on all power rails, ESD protection on I/O lines, CAN termination resistor footprints, motor connector with current rating, SWD debug header. Complete PCB layout with design rule check passing. Document thermal management approach for DRV8871.
5.4	Documentation Package	Complete all documentation deliverables listed in Section 15. Each document shall be reviewed for completeness, accuracy, and professional presentation before final commit.
5.5	Demo Video Recording	Record the demonstration sequence per Section 12. Video requirements: minimum 1080p, clear audio narration explaining each step, oscilloscope and logic analyzer screens visible and readable, OLED display visible. Edit to remove dead time. Total duration target: 8–12 minutes.

Phase Exit Criteria:

- All 15 demonstration steps execute successfully in sequence.
- All required debug evidence collected and annotated.
- PCB schematic DRC passes with zero errors.
- All documentation deliverables complete.
- Demo video recorded, edited, and uploaded.

7. RTOS DESIGN AND TASK SPECIFICATION

All tasks shall be implemented with defined periods, priorities, stack sizes, and inter-task communication mechanisms. Task priorities follow the rate-monotonic scheduling (RMS) principle: shorter period → higher priority. Stack sizes shall be verified with uxTaskGetStackHighWaterMark and documented in the validation report.

Task Name	Priority	Period	Stack	Responsibilities
Sensor Task	High — 3	10 ms	512 B	Read INA219 (current, voltage). Read RPM counter. Write to SensorDataQueue. Never blocks on queue send — use overwrite.
Control Task	High — 3	10 ms	512 B	Read SensorDataQueue. Execute PID calculation. Write PWM duty cycle via BSP. Write telemetry to TelemetryQueue.
Fault Task	Critical — 4	20 ms	512 B	Monitor sensor readings against fault thresholds. On fault: stop motor immediately, pend FaultEventQueue, set fault LED.
CAN Task	Medium — 2	Event	512 B	Blocked on CAN RX semaphore (MCP2515 INT). Parse received frame. Post motor command to MotorCmdQueue. Transmit status frames at 100 ms interval.
Logger Task	Low — 1	Event	768 B	Blocked on LogQueue. Dequeue log records. Write to W25Q128 flash. Handle sector erase when log wraps. Never preempts time-critical tasks.
Display Task	Low — 1	100 ms	1024 B	Read TelemetryQueue. Render OLED framebuffer. Flush to display. Handle menu navigation from encoder events.
CLI Task	Low — 1	Event	1024 B	UART receive interrupt deposits chars to RxQueue. CLI task parses commands: log dump, fault clear, telemetry print, PID gain adjust.

7.2 Inter-Task Communication

Object	Type	Depth	Producer → Consumer	Purpose
SensorDataQueue	Queue	1 item	Sensor Task → Control Task, Fault Task	Latest sensor snapshot. Single-item queue with overwrite semantics.
TelemetryQueue	Queue	1 item	Control Task → Display Task	Telemetry snapshot for display. Overwrite on full.
MotorCmdQueue	Queue	4 items	CAN Task → Control Task	Incoming motor speed commands from CAN.
FaultEventQueue	Queue	8 items	Fault Task → Logger Task	Fault records pending flash write.
LogQueue	Queue	16 items	All tasks → Logger Task	General event log records.
CANRxSemaphore	Binary Sem	N/A	ISR (MCP2515 INT) → CAN Task	Signals CAN frame available in MCP2515 RX buffer.

Object	Type	Depth	Producer → Consumer	Purpose
FaultMutex	Mutex	N/A	Fault Task, CLI Task	Protects fault status register from concurrent access.

8. DRIVER DEVELOPMENT SPECIFICATION

Each driver module shall expose a C header interface with the functions listed below. All driver functions shall return a status code (DRIVER_OK, DRIVER_ERR_TIMEOUT, DRIVER_ERR_COMM, DRIVER_ERR_INVALID). No driver function shall block for more than the timeout period specified. Drivers shall be testable in isolation using the UART debug console without requiring higher layers.

INA219 Driver

Header: ina219.h

Function Signature	Behavior
INA219_Status_t INA219_Init(INA219_Handle_t *h)	Initialize device. Configure gain, PGA, and calibration register. Verify device responds on I2C.
INA219_Status_t INA219_ReadCurrent(INA219_Handle_t *h, int32_t *mA)	Read shunt voltage register, apply calibration, return current in milliamperes.
INA219_Status_t INA219_ReadBusVoltage(INA219_Handle_t *h, uint32_t *mV)	Read bus voltage register, return in millivolts.
INA219_Status_t INA219_ReadPower(INA219_Handle_t *h, uint32_t *mW)	Read power register, return in milliwatts.

MCP2515 CAN Driver

Header: mcp2515.h

Function Signature	Behavior
MCP_Status_t MCP2515_Init(MCP_Handle_t *h, uint32_t bitrate)	Initialize MCP2515 via SPI. Configure bit timing for specified bitrate. Set operating mode to normal.
MCP_Status_t MCP2515_SendFrame(MCP_Handle_t *h, CAN_Frame_t *frame)	Load frame into TX buffer. Request transmission. Wait for TX pending flag clear (timeout 10 ms).
MCP_Status_t MCP2515_ReceiveFrame(MCP_Handle_t *h, CAN_Frame_t *frame)	Read frame from RX buffer. Clear RX flag. Return frame data.
MCP_Status_t MCP2515_SetFilter(MCP_Handle_t *h, uint8_t filter_n, uint32_t id)	Configure acceptance filter n with specified 11-bit or 29-bit ID.

Function Signature	Behavior
MCP_Status_t MCP2515_GetStatus(MCP_Handle_t *h, uint8_t *status)	Read MCP2515 status register. Return TX/RX buffer status.

W25Q128 Flash Driver

Header: flash.h

Function Signature	Behavior
Flash_Status_t Flash_Init(Flash_Handle_t *h)	Initialize SPI. Read JEDEC ID (0xEF4018). Verify identity. Return error if device not found.
Flash_Status_t Flash_ReadPage(Flash_Handle_t *h, uint32_t addr, uint8_t *buf, uint16_t len)	Read len bytes starting at addr. No erase required.
Flash_Status_t Flash_WritePage(Flash_Handle_t *h, uint32_t addr, uint8_t *buf, uint16_t len)	Write up to 256 bytes. Issue Write Enable before, poll WIP bit until complete (timeout 5 ms).
Flash_Status_t Flash_EraseSector(Flash_Handle_t *h, uint32_t sector_addr)	Erase 4 KB sector. Poll WIP bit until complete (timeout 400 ms).
Flash_Status_t Flash_ChipErase(Flash_Handle_t *h)	Erase entire chip. Poll WIP bit (timeout 200 s). Use only during factory reset.

OLED Display Driver

Header: oled.h

Function Signature	Behavior
OLED_Status_t OLED_Init(void)	Send SSD1306 initialization sequence over SPI. Set contrast, scan direction, display on.
void OLED_Clear(void)	Zero the in-memory framebuffer.
void OLED_DrawString(uint8_t x, uint8_t y, const char *str, Font_t *font)	Render string to framebuffer at pixel position (x, y).
void OLED_DrawRect(uint8_t x, uint8_t y, uint8_t w, uint8_t h, uint8_t fill)	Draw rectangle outline or filled to framebuffer.
void OLED_Flush(void)	Transmit entire 128×64 framebuffer to SSD1306 via SPI.

9. PID CONTROL SYSTEM DESIGN

9.1 Control Objective

The PID controller shall regulate motor shaft RPM to a commanded setpoint. The controller output is PWM duty cycle (0–100%). The process variable is measured RPM from the tachometer/encoder feedback. The controller shall execute in the Control Task at a fixed 10 ms sample period.

9.2 Discrete-Time PID Equation

$$u[k] = K_p * e[k] + K_i * dt * \text{sum}(e[0..k]) + K_d/dt * (e[k] - e[k-1])$$

where:

$e[k]$ = setpoint[k] - measured_rpm[k] (error at sample k)

$u[k]$ = controller output (PWM duty cycle, 0.0 to 1.0)

K_p, K_i, K_d = proportional, integral, derivative gains

$dt = 0.010$ s (10 ms sample period)

Derivative filter (N = filter coefficient, typically 5–20):

$$d_filtered[k] = N/(N+1) * d_filtered[k-1] + N*K_d/(N+1) * (e[k]-e[k-1])/dt$$

Anti-windup (integrator clamping):

If $u[k] > \text{output_max}$ OR $u[k] < \text{output_min}$: freeze integrator accumulation

9.3 Tuning Procedure

Step 1	Set $K_i = 0$, $K_d = 0$. Increase K_p until oscillation begins. Record as $K_{p_critical}$.
Step 2	Set $K_p = 0.5 \times K_{p_critical}$. Observe step response. Should be fast with some overshoot.
Step 3	Add K_i slowly until steady-state error is eliminated without inducing oscillation.
Step 4	Add K_d if overshoot is excessive. Do not over-tune derivative — it amplifies noise.
Step 5	Verify anti-windup by commanding large step (0 → 3000 RPM). Recovery should be clean.
Step 6	Document final K_p, K_i, K_d values in validation report with step response screenshot.

9.4 Performance Requirements

Parameter	Requirement	Measurement Method
Rise Time (0→90% RPM)	< 2.0 seconds	Measured on oscilloscope or UART data log
Overshoot	< 15%	Of commanded RPM setpoint
Steady-State Error	< ±5% RPM	Under constant no-load conditions
Load Disturbance Recovery	< 2.0 seconds	From manual shaft load application to within ±5%
Output Saturation Handling	No windup	Integrator freezes during output saturation

10. CAN PROTOCOL SPECIFICATION

10.1 Network Parameters

Bit Rate	500 kbps
Frame Format	CAN 2.0B Standard (11-bit identifiers)
Termination	120 Ω at each bus end (2 resistors total)
Transceiver	TJA1050 or SN65HVD230
Max Nodes	2 (motor controller + supervisory node or PC interface)

10.2 Message Definitions

ID (hex)	Name	Direction	DL C	Payload Description
0x100	MOTOR_CMD	Supervisory → Motor Ctrl	8	B0-B1: Target RPM (uint16, big-endian). B2: Command flags (bit0=start, bit1=stop, bit2=fault_clear). B3: Checksum (XOR B0-B2). B4-B7: Reserved.
0x101	MOTOR_STATUS	Motor Ctrl → Supervisory	8	B0-B1: Actual RPM (uint16). B2-B3: Current mA (uint16). B4: State (0=IDLE,1=RUN,2=FAULT). B5: Fault code. B6: Checksum (XOR B0-B5). B7: Reserved.
0x102	FAULT_EVENT	Motor Ctrl → Supervisory	4	B0: Fault type. B1: Fault severity. B2-B3: Timestamp low word (ms since boot).
0x1FF	HEARTBEAT	Both nodes, 1 Hz	1	B0: Node ID (0x01 = motor ctrl, 0x02 = supervisory). Absence triggers communication timeout fault.

11. FAULT MANAGEMENT SYSTEM

11.1 Fault Philosophy

All faults shall be treated as unrecoverable without explicit operator action. On any fault activation: the motor shall be stopped immediately (PWM duty cycle to 0%), the fault LED shall be illuminated, the fault shall be logged to flash with timestamp, and the system state machine shall transition to the FAULT state. The system shall not restart the motor until the operator explicitly clears the fault via encoder button press or CAN FAULT_CLEAR command.

11.2 Fault Definitions

ID	Name	Severity	Trigger Condition	Response Action
F-01	OVERCURRENT	Critical	INA219 current reading > 3000 mA for > 100 ms	Stop motor immediately. Log fault. Enter FAULT state.
F-02	UNDERVOLTAGE	Critical	INA219 bus voltage < 5500 mV for > 500 ms	Stop motor. Log fault. Enter FAULT state.
F-03	SENSOR_TIMEOUT	Critical	INA219 I2C read fails or times out for > 3 consecutive reads	Stop motor. Log fault. Enter FAULT state.
F-04	CAN_TIMEOUT	Warning	No CAN frame received for > 5 s when operating in CAN mode	Stop motor. Log fault. Enter FAULT state. (Prevents runaway on cable disconnect.)
F-05	RPM_DIVERGENCE	Critical	Measured RPM deviates > 20% from commanded for > 3 s	Stop motor. Log fault. Indicates motor stall or feedback failure.
F-06	DRIVER_FAULT	Critical	DRV8871 nFAULT pin asserted	Stop motor immediately (hardware-level). Log fault. Enter FAULT state.
F-07	FLASH_FAIL	Warning	W25Q128 write or read fails after 3 retries	Log fault to UART only. Continue operation with logging disabled.

11.3 Flash Log Record Format

Each log record is 32 bytes with the following structure:

Offset	Size	Field	Description
0x00	4 B	timestamp_ms	Milliseconds since system boot (uint32)
0x04	1 B	event_type	0x01=FAULT, 0x02=INFO, 0x03=WARNING, 0x04=STATE
0x05	1 B	fault_code	Fault ID (F-01 through F-07), 0x00 if not fault
0x06	1 B	severity	0=INFO, 1=WARNING, 2=CRITICAL
0x07	1 B	motor_state	State machine state at time of event
0x08	2 B	current_mA	INA219 reading at time of event (uint16)
0x0A	2 B	voltage_mV	INA219 bus voltage at time of event (uint16)
0x0C	2 B	rpm_actual	Measured RPM at time of event (uint16)

```
0x0E 2 B rpm_setpoint Commanded RPM at time of event (uint16)
0x10 2 B pwm_duty PWM duty cycle × 100 (0-10000) (uint16)
0x12 10 B reserved Zero-padded
0x1C 2 B record_count Monotonic record counter (uint16)
0x1E 2 B checksum XOR of bytes 0x00-0x1D (uint16)
```

12. VALIDATION AND TEST PLAN

12.1 Required Demonstration Sequence

The following sequence shall be executed in order for the final demonstration video. Each step shall be narrated on camera. Steps must execute reliably without pre-positioning.

Step	Action	Expected Outcome / Narration Guide
Step 1	Power-On Self Test	Apply power. Observe OLED boot screen with firmware version. Observe UART console power-on message. Verify no fault LED illuminated.
Step 2	OLED Menu Navigation	Rotate encoder to scroll through main menu items. Press button to enter submenus. Demonstrate all menu screens accessible and correctly labeled.
Step 3	Set Target RPM	Navigate to RPM setpoint screen. Rotate encoder to set target to 1500 RPM. Confirm value displayed correctly.
Step 4	Start Motor	Press encoder button to start motor. Observe motor begin spinning. Observe OLED live RPM reading rising to setpoint.
Step 5	Closed-Loop RPM Control Demonstration	Allow motor to reach steady-state. Point camera at OLED showing RPM within $\pm 5\%$ of 1500 RPM. Comment on PID response.
Step 6	Load Disturbance Application	Apply light hand friction to motor shaft. Narrate that PID is compensating. Show RPM recovery on OLED within 2 seconds.
Step 7	Current Limiting Observation	Show INA219 current reading increasing under load. Reference oscilloscope capture of startup current.
Step 8	Trigger Overcurrent Fault	Stall the motor (hard stop). Observe fault LED illuminate within 100 ms. Observe motor stop. Observe OLED display FAULT state.
Step 9	Fault LED and State Indication	Show fault LED illuminated. Show OLED fault screen with fault code F-01 OVERCURRENT.
Step 10	Fault Log Retrieval	Open serial terminal. Send log dump command via CLI. Show timestamped fault record in UART output.
Step 11	Sensor Disconnection Test	Clear fault. Restart motor. Disconnect INA219 I2C wire while running. Show F-03 SENSOR_TIMEOUT fault and safe shutdown.
Step 12	Safe Shutdown Demonstration	Reconnect sensor. Clear fault. Demonstrate controlled stop command via encoder. Motor decelerates and stops cleanly.
Step 13	CAN Command Transmission	Send MOTOR_CMD frame from PC Python script or second node. Show logic analyzer capture of CAN frame on bus.
Step 14	Motor Response to CAN Command	Show motor starting and reaching RPM commanded via CAN. Show MOTOR_STATUS frames being transmitted on bus.
Step 15	Debug Evidence Review	Pan to oscilloscope showing PWM waveform. Show logic analyzer SPI and I2C transaction captures. Show JTAG task list in debugger.

13. DEBUG EVIDENCE REQUIREMENTS

All debug evidence shall be saved as high-resolution images or tool-native export files. Each capture shall be annotated in the image or in an accompanying markdown file identifying: the signals shown, the event captured, the measurement values, and what the capture proves about system behavior.

ID	Tool	Capture Subject	Description and Acceptance Criteria
OSC-01	Oscilloscope	PWM Waveform	TIM1 PWM output at 50% duty cycle. Show 20 kHz frequency, correct high/low times, clean edges.
OSC-02	Oscilloscope	Motor Startup Current	INA219 analog output or shunt voltage during motor start. Show inrush current peak and settling.
OSC-03	Oscilloscope	Current Limiting Event	Capture overcurrent fault trigger moment. Show current threshold crossing and PWM cutoff.
OSC-04	Oscilloscope	PID Step Response	RPM vs time from 0 to setpoint via DAC output or UART data exported to plot. Show rise time and overshoot.
LA-01	Logic Analyzer	SPI Flash Transaction	Full SPI frame: CS assert, clock, MOSI command bytes, MISO read data, CS deassert. Decode protocol.
LA-02	Logic Analyzer	I2C INA219 Transaction	I2C START, address, register write, repeated START, data read, STOP. Decode and verify register values.
LA-03	Logic Analyzer	CAN Frame Capture	Full CAN MOTOR_CMD frame. Show ID field, DLC, data bytes, CRC, ACK. Decode to hex.
LA-04	Logic Analyzer	SPI MCP2515 Transaction	SPI frames to/from MCP2515 during CAN send. Correlate with LA-03 CAN frame.
JTAG-01	JTAG/SWD	FreeRTOS Task List	Debugger task view showing all 7 tasks, their states, priorities, and stack high watermarks.
JTAG-02	JTAG/SWD	Live Variable Watch	Debugger watch window showing PID error, output, setpoint, and measured RPM during motor run.
JTAG-03	JTAG/SWD	Queue Inspection	Show SensorDataQueue contents with sensor snapshot values visible in debugger memory view.
JTAG-04	JTAG/SWD	Breakpoint Hit	Breakpoint in fault_manager.c fault detection path. Show call stack at breakpoint.

14. RISK REGISTER

The following risks are identified at project outset. Each risk has a defined mitigation strategy. Risks shall be reviewed at the end of each phase and updated as new information becomes available.

ID	Level	Risk	Description	Mitigation
R-01	HIGH	PID Tuning Difficulty	Motor behavior is nonlinear or noisy, making PID tuning unstable or time-consuming.	Start tuning at very low RPM setpoints. Log RPM data to UART for analysis. Implement derivative filter. Budget 3–5 days.
R-02	HIGH	Hardware Damage	DRV8871 or INA219 destroyed by wiring error or overcurrent during development.	Order 2× of each IC. Triple-check power connections before power-on. Use current-limited bench supply. Verify with multimeter before applying motor power.
R-03	MEDIUM	MCP2515 CAN Bringup	SPI timing or bit-timing configuration issues prevent CAN communication.	Validate SPI communication with register read-back before attempting CAN frames. Use logic analyzer from first CAN attempt. Have Python can-utils as known-good CAN source.
R-04	MEDIUM	STM32CubeMX Regeneration Breaks Code	Modifying CubeMX config regenerates code and overwrites manual edits.	Keep all manual code in USER CODE BEGIN/END sections. Document this in coding standards. Test regeneration in a branch before merging.
R-05	MEDIUM	Flash Log Wear	Repeated testing erases same flash sectors, reducing endurance.	Implement circular buffer with head pointer stored in last valid sector. Test on a dedicated "test sector" during development.
R-06	LOW	RPM Measurement Noise	Encoder or Hall sensor signal has noise causing erratic RPM readings.	Implement software low-pass filter on RPM measurement. Use hardware Schmidt trigger input if available. Increase measurement window to 50–100 ms.
R-07	LOW	Schedule Overrun	Project takes significantly longer than estimated.	Phases are designed so each boundary produces a demonstrable artifact. A project ending at Phase 3 or 4 is still a strong portfolio item.

15. FINAL DELIVERABLES CHECKLIST

The following deliverables constitute the complete project output. All items shall be committed to the GitHub repository under the Docs/ directory and linked from the README. The README shall serve as the project landing page and shall present the project professionally to a technical hiring audience.

ID	Deliverable	Location	Content Requirements
D-01	Firmware Repository	GitHub	Complete, compiling firmware with all features. Clean commit history. Professional README.
D-02	Requirements Specification	Docs/reqs/	Formal requirements document. All requirements numbered (REQ-xx). Traceable to test cases.
D-03	Architecture Document	Docs/arch/	System block diagram, firmware layer diagram, module dependency map, design decisions rationale.
D-04	Driver Documentation	Docs/drivers/	API reference for each driver module. Register maps for INA219, MCP2515, W25Q128.
D-05	RTOS Design Document	Docs/rtos/	Task table, IPC diagram, stack sizing rationale, measured watermarks.
D-06	CAN Protocol Specification	Docs/can/	Message table, frame format diagrams, timing requirements, example frames.
D-07	PCB Schematic	Docs/hw/	KiCad schematic PDF export. Net list. BOM. Design review checklist completed.
D-08	PCB Layout	Docs/hw/	KiCad layout Gerber export. DRC report showing zero errors. Layer stack description.
D-09	Validation Report	Docs/validation/	Test results for all success criteria SC-01 through SC-14. PID tuning data. Pass/fail status.
D-10	Debug Report	Docs/debug/	All debug evidence items DE-01 through DE-12. Annotated captures. Narrative explaining each.
D-11	Demo Video	YouTube/Portfolio	Minimum 1080p. 8–12 minutes. Narrated. All 15 demonstration steps. Linked from README.
D-12	Portfolio Screenshots	Docs/media/	OLED display, oscilloscope, logic analyzer, JTAG, PCB render. Minimum 1920×1080 resolution.

A Note on Documentation Quality

The documentation package is not a formality — it is half the portfolio artifact.

A hiring engineer reading your architecture document is evaluating whether you think like a professional.

Write every document as if it will be reviewed by a senior engineer who will ask you to defend every decision.

If you cannot explain why you made a design choice, reconsider the choice before documenting it.